

```
import itertools
import numpy as np
```

```
def bipolar(net, lambdaa = 1):
    return (2/(1+np.exp(-lambdaa*net))) - 1
```

```
def unipolar(net, lambdaa = 0.3):
    return (1/(1+np.exp(-lambdaa*net)))
```

```
def bipolar_fdash(y_pred):
    return (1/2)*(1-(y_pred**2))
```

```
def unipolar_fdash(y_pred):
    return (y_pred*(1-y_pred))
```

```
def predict(X, weights, type):
    net = np.dot(X, weights)
    if type == 'unipolar':
        return unipolar(net)
    elif type == 'bipolar':
        return bipolar(net)
```

```
def calculate_r(yi, y_pred, type):
    r = yi - y_pred
    if type == 'unipolar':
        return r * unipolar_fdash(y_pred)
    elif type == 'bipolar':
        return r * bipolar_fdash(y_pred)
```

```
def train(X, y, weights, type, epochs=500, c=0.5):
    for epoch in range(epochs):
        loss = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights, type)
            r = calculate_r(yi, y_pred, type)
            loss += abs(r)
            delta_w = c*r*Xi
            weights += delta_w

        if (epoch + 1) % 50 == 0:
            print(f"Weights after epoch {epoch + 1}: ", weights)
        if loss == 0:
            break

    print('Learned Weights: ', weights)
    weights = weights.reshape((n+1, 1))
    test(X, y, weights, type)
```

```
def test(X, y, learned_weights, type):
    nets = np.dot(X, learned_weights).flatten()
    print('Actual values: ', y)

    if type == 'unipolar':
        y_pred = np.array([unipolar(net) for net in nets])
        print("Predicted values: ", y_pred)
    else:
        y_pred = np.array([bipolar(net) for net in nets])
        print("Predicted values: ", y_pred)
```

```
n = int(input("Enter the number of bits: "))
X = np.array([list(i) + [1] for i in itertools.product([0,1], repeat=n)])

weights = input(f'Enter initial {n} weights and 1 bias : ')
weights = np.array([float(weight) for weight in weights.split()], dtype='longdouble')
```

```
↻ Enter the number of bits: 2
Enter initial 2 weights and 1 bias : 1 1 -1.5
```

```
# 1) AND GATE UNIPOLAR
print('---- AND GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'unipolar')
```

```

---- AND GATE USING PERCEPTRON ----
Weights after epoch 50: [ 2.06884716  2.04969855 -4.15009132]
Weights after epoch 100: [ 3.21454526  3.18761228 -5.59950151]
Weights after epoch 150: [ 4.12101034  4.09107424 -6.8015584 ]
Weights after epoch 200: [ 4.86924412  4.83886183 -7.83869085]
Weights after epoch 250: [ 5.51133257  5.48178246 -8.75087868]
Weights after epoch 300: [ 6.07637942  6.04823695 -9.56539652]
Weights after epoch 350: [ 6.58209923  6.55557193 -10.30114698]
Weights after epoch 400: [ 7.04019002  7.01530007 -10.97171574]
Weights after epoch 450: [ 7.45889716  7.43557807 -11.587263 ]
Weights after epoch 500: [ 7.84433339  7.82248044 -12.1556508 ]
Learned Weights: [ 7.84433339  7.82248044 -12.1556508 ]
Actual values: [0 0 0 1]
Predicted values: [0.02541443 0.21417325 0.21527869 0.74141746]

```

```

# 2) OR GATE UNIPOLAR
print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = 0
train(X, y, weights.copy(), 'unipolar')

```

```

---- OR GATE USING PERCEPTRON ----
Weights after epoch 50: [ 3.63909071  3.6534463 -0.01317792]
Weights after epoch 100: [ 4.85431882  4.87106131 -0.68777983]
Weights after epoch 150: [ 5.83760728  5.85484642 -1.36782382]
Weights after epoch 200: [ 6.68438769  6.70133652 -1.93282344]
Weights after epoch 250: [ 7.42466934  7.44093767 -2.40177081]
Weights after epoch 300: [ 8.07798159  8.09339764 -2.79928944]
Weights after epoch 350: [ 8.65953452  8.67404876 -3.14299425]
Weights after epoch 400: [ 9.1814258  9.19505335 -3.44493946]
Weights after epoch 450: [ 9.65329659  9.6660841 -3.71359625]
Weights after epoch 500: [10.08285878 10.09486608 -3.95512957]
Learned Weights: [10.08285878 10.09486608 -3.95512957]
Actual values: [0 1 1 1]
Predicted values: [0.23387853 0.86317574 0.86274975 0.99236068]

```

```

# 3) NOR GATE UNIPOLAR
print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'unipolar')

```

```

---- NOR GATE USING PERCEPTRON ----
Weights after epoch 50: [-1.79174464 -1.80368285 -2.14901381]
Weights after epoch 100: [-3.15956598 -3.1744099 -0.92495224]
Weights after epoch 150: [-4.33371745 -4.35027974  0.17307251]
Weights after epoch 200: [-5.36795999 -5.38526488  1.00942644]
Weights after epoch 250: [-6.27166612 -6.28891867  1.65439941]
Weights after epoch 300: [-7.06176753 -7.07847063  2.17302379]
Weights after epoch 350: [-7.75674595 -7.77264756  2.60510771]
Weights after epoch 400: [-8.37291534 -8.38791913  2.97454145]
Weights after epoch 450: [-8.92368652 -8.93778341  3.29647892]
Weights after epoch 500: [-9.41983492 -9.4330606  3.58113845]
Learned Weights: [-9.41983492 -9.4330606  3.58113845]
Actual values: [1 0 0 0]
Predicted values: [0.74542168 0.1473452 0.14784438 0.01013547]

```

```

# 4) NAND GATE UNIPOLAR
print('---- NAND GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[-1] = 0
train(X, y, weights.copy(), 'unipolar')

```

```

---- NAND GATE USING PERCEPTRON ----
Weights after epoch 50: [-0.28234401 -0.26244769  2.3176296 ]
Weights after epoch 100: [-1.89630502 -1.86825878  4.03801651]
Weights after epoch 150: [-3.1066732 -3.07466593  5.46235154]
Weights after epoch 200: [-4.03497992 -4.00196397  6.68258568]
Weights after epoch 250: [-4.79639096 -4.76405692  7.73498854]
Weights after epoch 300: [-5.44772644 -5.41688584  8.65897372]
Weights after epoch 350: [-6.01975913 -5.99072714  9.48285662]
Weights after epoch 400: [-6.5310192 -6.50385404 10.22625855]
Weights after epoch 450: [-6.99365328 -6.96828897 10.90322307]
Weights after epoch 500: [-7.41617549 -7.3924919 11.52421072]
Learned Weights: [-7.41617549 -7.3924919 11.52421072]
Actual values: [1 1 1 0]
Predicted values: [0.96944701 0.77547969 0.7742402 0.27183409]

```

```

# 5) AND GATE BIPOLAR
print('---- AND GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'bipolar')

```

```

---- AND GATE USING PERCEPTRON ----
Weights after epoch 50: [ 3.0647508  3.00073714 -4.55023391]
Weights after epoch 100: [ 3.78249802  3.74775663 -5.6901228 ]
Weights after epoch 150: [ 4.22530613  4.20182914 -6.37837491]
Weights after epoch 200: [ 4.54385074  4.52621902 -6.86849581]
Weights after epoch 250: [ 4.79186665  4.77778938 -7.24789224]
Weights after epoch 300: [ 4.99455859  4.98286282 -7.5567974 ]
Weights after epoch 350: [ 5.16572803  5.15573565 -7.81698281]
Weights after epoch 400: [ 5.313736  5.30502068 -8.04153109]
Weights after epoch 450: [ 5.44402351  5.4363002 -8.23890561]
Weights after epoch 500: [ 5.56032592  5.55339487 -8.41489048]
Learned Weights: [ 5.56032592  5.55339487 -8.41489048]
Actual values: [-1 -1 -1 1]
Predicted values: [-0.99955701 -0.89181975 -0.8911083  0.87391518]

```

```

# 6) OR GATE BIPOLAR
print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = -1
train(X, y, weights.copy(), 'bipolar')

```

```

---- OR GATE USING PERCEPTRON ----
Weights after epoch 50: [ 3.61579381  3.65075785 -1.49469801]
Weights after epoch 100: [ 4.39603673  4.41601861 -1.91714424]
Weights after epoch 150: [ 4.86066098  4.87445901 -2.16197481]
Weights after epoch 200: [ 5.18925445  5.19974165 -2.3331059 ]
Weights after epoch 250: [ 5.44258714  5.45102536 -2.46415489]
Weights after epoch 300: [ 5.64831248  5.65536211 -2.57010427]
Weights after epoch 350: [ 5.82127992  5.82732825 -2.65889964]
Weights after epoch 400: [ 5.97036338  5.97565639 -2.73524918]
Weights after epoch 450: [ 6.10127968  6.10598311 -2.80216732]
Weights after epoch 500: [ 6.21792319  6.2221539 -2.8616978 ]
Learned Weights: [ 6.21792319  6.2221539 -2.8616978 ]
Actual values: [-1  1  1  1]
Predicted values: [-0.89184043  0.93289114  0.93261621  0.99986159]

```

```

# 7) NOR GATE BIPOLAR
print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'bipolar')

```

```

---- NOR GATE USING PERCEPTRON ----
Weights after epoch 50: [-2.97472076 -3.02599701  1.12991304]
Weights after epoch 100: [-4.09674733 -4.12183424  1.75713199]
Weights after epoch 150: [-4.66914899 -4.68528746  2.06152202]
Weights after epoch 200: [-5.04938819 -5.0611983  2.26044009]
Weights after epoch 250: [-5.33278769 -5.34207119  2.40744186]
Weights after epoch 300: [-5.55811004 -5.56574459  2.52369852]
Weights after epoch 350: [-5.74483009 -5.75130643  2.61968355]
Weights after epoch 400: [-5.90408145 -5.90970085  2.70132494]
Weights after epoch 450: [-6.04281265 -6.04777298  2.77229623]
Weights after epoch 500: [-6.16564539 -6.17008345  2.83502778]
Learned Weights: [-6.16564539 -6.17008345  2.83502778]
Actual values: [ 1 -1 -1 -1]
Predicted values: [ 0.88907914 -0.93122408 -0.93092874 -0.99985041]

```

```

# 8) NAND GATE BIPOLAR
print('---- NAND GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[-1] = -1
train(X, y, weights.copy(), 'bipolar')

```

```

---- NAND GATE USING PERCEPTRON ----
Weights after epoch 50: [-2.56383172 -2.46880693  3.73168195]
Weights after epoch 100: [-3.53062038 -3.4873509  5.29400115]
Weights after epoch 150: [-4.05911804 -4.03189379  6.12113385]
Weights after epoch 200: [-4.42052894 -4.4008221  6.67916638]
Weights after epoch 250: [-4.69411708 -4.67873006  7.0985642 ]
Weights after epoch 300: [-4.91373963 -4.90114501  7.4337404 ]
Weights after epoch 350: [-5.0969204 -5.08627428  7.71246036]
Weights after epoch 400: [-5.25387996 -5.24466846  7.95076581]
Weights after epoch 450: [-5.39108948 -5.38297711  8.15874559]
Weights after epoch 500: [-5.51290024 -5.50565615  8.34314935]
Learned Weights: [-5.51290024 -5.50565615  8.34314935]
Actual values: [ 1  1  1 -1]
Predicted values: [ 0.99952407  0.88933716  0.88857742 -0.87111956]

```

